



CinemaGo

Votre Compagnon Cinéma

Rapport Technique

Application Mobile Android

Réalisé par : Aymen Medejel

Encadrant : Bousmah

Année universitaire : 2025-2026

Rapport Technique — CinemaGo

Projet Android développé en Java

Table des matières

1	Introduction	3
1.1	Présentation du projet	3
1.2	Contexte et motivation	3
1.3	Objectifs	3
2	Architecture du Projet	4
2.1	Vue d'ensemble	4
2.2	Structure des packages	4
2.3	Hierarchie des composants	4
2.4	Flux de données	5
3	Technologies Utilisées	6
3.1	Stack technique	6
3.2	API Externes	6
3.2.1	TMDB API (The Movie Database)	6
3.2.2	Google Maps & Places API	7
3.2.3	Gemini API (Google AI)	7
4	Fonctionnalités Détaillées	8
4.1	Authentification Firebase	8
4.2	Écran d'accueil — Films Tendances	8
4.3	Recherche Vocale et Textuelle	8
4.4	Carte et Cinémas Proches	9
4.5	Chatbot IA — CineBot	9
4.6	Caméra et Upload Firebase	9
4.7	Favoris	10
5	Modèles de Données	11
5.1	Classe Movie	11
5.2	Classe Cinema	11
5.3	Classe FavoriteMovie	12
6	Interface Utilisateur	13
6.1	Thème visuel	13
6.2	Navigation principale	13
6.3	Layouts principaux	13

7	Firestore et Sécurité	15
7.1	Architecture Firestore	15
7.2	Structure Firestore	15
7.3	Gestionnaire Firestore Singleton	15
8	Permissions Android	17
8.1	Permissions requises	17
8.2	Gestion dynamique des permissions	17
9	Guide d'Installation	18
9.1	Prérequis	18
9.2	Étapes de configuration	18
9.3	Variables de configuration	18
10	Conclusion et Perspectives	20
10.1	Bilan du projet	20
10.2	Perspectives d'amélioration	20
10.3	Répartition du code	21
	Références	22

Chapitre 1

Introduction

1.1 Présentation du projet

CinemaGo est une application mobile Android développée en Java avec des layouts XML. Elle permet aux utilisateurs de découvrir des films, de trouver des cinémas à proximité, de discuter avec un chatbot intelligent, et de sauvegarder leurs films favoris.

Objectif du projet

Créer une application cinéma simple mais professionnelle, adaptée à un projet étudiant, intégrant les technologies modernes du développement Android : Firebase, TMDB API, Google Maps, CameraX et l'intelligence artificielle Gemini.

1.2 Contexte et motivation

L'industrie du cinéma est en pleine évolution numérique. Les utilisateurs souhaitent accéder facilement aux informations sur les films, trouver les salles proches de chez eux et obtenir des recommandations personnalisées. CinemaGo répond à ces besoins en centralisant toutes ces fonctionnalités dans une seule application.

1.3 Objectifs

Les objectifs principaux du projet sont :

- Permettre la recherche de films par texte et par voix
- Afficher les films tendances via l'API TMDB
- Localiser les cinémas proches grâce au GPS
- Offrir un chatbot IA pour des recommandations de films
- Permettre la prise de photos avec upload Firebase
- Gérer les films favoris avec authentification utilisateur

Chapitre 2

Architecture du Projet

2.1 Vue d'ensemble

L'application suit une architecture en couches inspirée du patron MVC (**Model-View-Controller**), adaptée au contexte Android. Elle est organisée en 7 packages distincts pour assurer une séparation claire des responsabilités.

Principe de l'architecture

Chaque couche a une responsabilité unique : les **Models** stockent les données, les **Adapters** les affichent en liste, les **Fragments** gèrent les écrans, et les **Activities** orchestrent l'ensemble.

2.2 Structure des packages

TABLE 2.1 – Organisation des packages du projet CinemaGo

Package	Fichiers	Rôle
activities	6 fichiers	Écrans complets de l'application
fragments	5 fichiers	Onglets du bas (Home, Search, Map, Chat, Profile)
adapters	4 fichiers	Affichage des listes RecyclerView
models	5 fichiers	Classes de données (Movie, Cinema, etc.)
api	2 fichiers	Client Retrofit + interface TMDB
firebase	1 fichier	Gestionnaire Auth, Firestore, Storage
chatbot	1 fichier	Intégration API Gemini
utils	4 fichiers	Outils transversaux (GPS, permissions, réseau)

2.3 Hiérarchie des composants

La hiérarchie complète de l'application est la suivante :

MainActivity (Activity principale)

HomeFragment	→ MovieAdapter	→ Movie
SearchFragment	→ MovieAdapter	→ Movie
MapFragment	→ CinemaAdapter	→ Cinema
ChatFragment	→ ChatAdapter	→ ChatMessage
ProfileFragment	→ FavoriteAdapter	→ FavoriteMovie

2.4 Flux de données

Le flux de données dans CinemaGo suit un chemin précis :

1. Le **Fragment** appelle l'API (TMDB ou Firebase)
2. L'API retourne des données JSON converties en **Models**
3. Le **Fragment** transmet la liste de Models à l'**Adapter**
4. L'**Adapter** inflat le layout XML et remplit chaque vue
5. L'utilisateur voit la liste à l'écran

Chapitre 3

Technologies Utilisées

3.1 Stack technique

TABLE 3.1 – Technologies et dépendances du projet

Technologie	Version	Usage
Java	8+	Langage principal
Android SDK	API 24–34	Plateforme cible
Firebase Auth	BOM 33.0	Authentification utilisateur
Firebase Firestore	BOM 33.0	Base de données NoSQL
Firebase Storage	BOM 33.0	Stockage des photos
Retrofit 2	2.9.0	Appels réseau REST
Gson	2.9.0	Sérialisation JSON
Glide	4.16.0	Chargement d'images
CameraX	1.3.3	Capture photos
Google Maps SDK	18.2.0	Affichage carte
Places API	3.4.0	Recherche cinémas
Gemini SDK	0.7.0	Chatbot IA
TMDB API	v3	Données films
SpeechRecognizer	Natif Android	Recherche vocale

3.2 API Externes

3.2.1 TMDB API (The Movie Database)

L'API TMDB fournit toutes les données relatives aux films : titres, affiches, notes, résumés, dates de sortie. Elle est utilisée pour :

- Récupérer les films tendances de la semaine
- Rechercher des films par titre

```
1 public interface TmdbApiService {
2     @GET("trending/movie/week")
3     Call<MovieResponse> getTrendingMovies(
4         @Query("api_key") String apiKey);
```

```
5      @GET("search/movie")
6      Call<MovieResponse> searchMovies(
7          @Query("api_key") String apiKey,
8          @Query("query") String query);
9
10 }
```

Listing 3.1 – Interface Retrofit pour TMDb

3.2.2 Google Maps & Places API

L'API Google Maps affiche une carte interactive avec les marqueurs des cinémas. L'API Places (New) recherche les cinémas dans un rayon de 5km autour de l'utilisateur.

3.2.3 Gemini API (Google AI)

L'API Gemini 1.5 Flash alimente le chatbot CineBot. Un prompt système définit le comportement du bot comme un expert en cinéma.

Chapitre 4

Fonctionnalités Détaillées

4.1 Authentification Firebase

Le système d'authentification utilise Firebase Auth avec email/mot de passe. Il comprend trois écrans :

- **SplashActivity** : Vérifie si l'utilisateur est déjà connecté au démarrage
- **LoginActivity** : Formulaire de connexion avec validation
- **RegisterActivity** : Création de compte avec confirmation du mot de passe

```
1 FirebaseManager.getInstance().login(email, password,
2     new FirebaseManager.AuthCallback() {
3         @Override
4         public void onSuccess(FirebaseUser user) {
5             startActivity(new Intent(this, MainActivity.class));
6             finish();
7         }
8         @Override
9         public void onError(String error) {
10            Toast.makeText(this, "Erreur : " + error,
11                Toast.LENGTH_LONG).show();
12        }
13    });
```

Listing 4.1 – Connexion avec Firebase Auth

4.2 Écran d'accueil — Films Tendances

HomeFragment charge les films tendances depuis TMDB et les affiche dans une grille 2 colonnes via MovieAdapter. Un bouton FAB permet d'accéder à la caméra.

4.3 Recherche Vocale et Textuelle

SearchFragment propose deux modes de recherche :

1. **Textuelle** : Un TextWatcher avec debounce de 600ms lance la recherche automatiquement après 3 caractères saisis

2. **Vocale** : L'Android `SpeechRecognizer` convertit la parole en texte qui est ensuite envoyé à TMDb

```
1 Intent intent = new Intent(RecognizerIntent.  
    ACTION_RECOGNIZE_SPEECH);  
2 intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL ,  
3     RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);  
4 intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE ,  
5     Locale.getDefault());  
6 speechRecognizer.startListening(intent);
```

Listing 4.2 – Démarrage de la reconnaissance vocale

4.4 Carte et Cinémas Proches

`MapFragment` combine plusieurs technologies :

- **FusedLocationProviderClient** : Récupère la position GPS en temps réel
- **Places API (New)** : Recherche les cinémas dans un rayon rectangulaire de 5km
- **Google Maps** : Affiche les marqueurs rouges pour chaque cinéma trouvé
- **Navigation Intent** : Permet d'ouvrir Google Maps pour l'itinéraire

Un style de carte sombre (`map_style_dark.json`) est appliqué pour correspondre au thème de l'application.

4.5 Chatbot IA — CineBot

Le chatbot utilise l'API Gemini 1.5 Flash. Un contexte système est injecté à chaque requête pour configurer le comportement du bot :

```
1 private static final String SYSTEM_CONTEXT =  
2     "You are CineBot, a friendly movie expert assistant " +  
3     "for the CinemaGo app. Help users discover movies, " +  
4     "get recommendations, learn about actors and directors. " +  
5     "Keep responses concise and engaging.";
```

Listing 4.3 – Prompt système du chatbot CineBot

`ChatAdapter` utilise deux types de vues distincts : `TYPE_USER` et `TYPE_AI`, avec des bulles de couleurs différentes.

4.6 Caméra et Upload Firebase

`CameraActivity` utilise `CameraX` pour :

1. Démarrer l'aperçu en temps réel dans un `PreviewView`
2. Capturer une photo via `ImageCapture`
3. Uploader le fichier sur Firebase Storage
4. Sauvegarder l'URL de téléchargement dans Firestore

```
1 imageCapture.takePicture(options,
2   ContextCompat.getMainExecutor(this),
3   new ImageCapture.OnImageSavedCallback() {
4       @Override
5       public void onImageSaved(OutputFileResults output) {
6           Uri savedUri = Uri.fromFile(photoFile);
7           uploadPhoto(savedUri); // Upload vers Firebase
8       }
9       @Override
10      public void onError(ImageCaptureException exception) {
11          Toast.makeText(context, "Erreur capture",
12              Toast.LENGTH_SHORT).show();
13      }
14  });
```

Listing 4.4 – Capture photo avec CameraX

4.7 Favoris

ProfileFragment affiche les films favoris de l'utilisateur stockés dans Firestore. Chaque favori est identifié par la clé composite `userId_movieId` pour éviter les doublons.

Chapitre 5

Modèles de Données

5.1 Classe Movie

TABLE 5.1 – Attributs de la classe Movie

Type	Attribut	Description
int	id	Identifiant TMDB unique
String	title	Titre du film
String	overview	Résumé du film
String	posterPath	Chemin relatif de l’affiche
String	backdropPath	Chemin relatif de l’image de fond
double	voteAverage	Note moyenne (0–10)
String	releaseDate	Date de sortie (AAAA-MM-JJ)

5.2 Classe Cinema

TABLE 5.2 – Attributs de la classe Cinema

Type	Attribut	Description
String	name	Nom du cinéma
String	address	Adresse complète
double	latitude	Coordonnée GPS latitude
double	longitude	Coordonnée GPS longitude
float	rating	Note Google (0–5)
String	placeId	Identifiant Google Places

5.3 Classe FavoriteMovie

TABLE 5.3 – Attributs de la classe FavoriteMovie (stockée dans Firestore)

Type	Attribut	Description
String	movieId	ID TMDB du film
String	title	Titre du film
String	posterUrl	URL complète de l’affiche
double	rating	Note du film
String	userId	UID Firebase de l’utilisateur

Chapitre 6

Interface Utilisateur

6.1 Thème visuel

L'application adopte un thème sombre inspiré des applications de streaming populaires.

TABLE 6.1 – Palette de couleurs de CinemaGo

Couleur	Code Hex	Usage
	#E50914	Couleur primaire, boutons, accents
	#F5C518	Notes, étoiles, liens
	#0F0F1A	Arrière-plan principal
	#1A1A2E	Cartes et surfaces

6.2 Navigation principale

La navigation est assurée par une `BottomNavigationView` avec 5 onglets :

TABLE 6.2 – Onglets de navigation

Icône	Onglet	Fragment associé
Maison	Accueil	HomeFragment
Loupe	Recherche	SearchFragment
Carte	Carte	MapFragment
Bulle	Chat	ChatFragment
Personne	Profil	ProfileFragment

6.3 Layouts principaux

Listing 6.1 – Extrait du layout `activity_main.xml`

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:background="@color/bg_dark">

    <FrameLayout
```

```
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"/>

        <com.google.android.material.bottomnavigation.
        BottomNavigationView
            android:id="@+id/bottom_nav"
            app:menu="@menu/bottom_nav_menu"
            app:itemIconTint="@drawable/bottom_nav_selector"/>
    </androidx.constraintlayout.widget.ConstraintLayout>
```

Chapitre 7

Firestore et Sécurité

7.1 Architecture Firebase

Firebase est le backend complet de CinemaGo. Il assure trois rôles distincts :

1. **Firebase Auth** : Gestion de l'identité utilisateur (connexion, inscription, déconnexion)
2. **Cloud Firestore** : Stockage des favoris et métadonnées des photos
3. **Firebase Storage** : Stockage des photos capturées avec la caméra

7.2 Structure Firestore

```
Firestore Database
favorites/
  {userId}_{movieId}
    movieId : "550"
    title   : "Fight Club"
    posterUrl : "https://..."
    rating  : 8.8
    userId  : "abc123"

userPhotos/
  {autoId}
    url      : "https://storage.firebase..."
    userId   : "abc123"
    timestamp : 1234567890
```

7.3 Gestionnaire Firebase Singleton

`FirebaseManager` suit le patron **Singleton** pour garantir une seule instance partagée dans toute l'application.

```
1 private static FirebaseManager instance;
2
3 public static synchronized FirebaseManager getInstance() {
```



```
4     if (instance == null) {  
5         instance = new FirebaseManager();  
6     }  
7     return instance;  
8 }
```

Listing 7.1 – Patron Singleton dans FirebaseManager

Chapitre 8

Permissions Android

8.1 Permissions requises

TABLE 8.1 – Permissions déclarées dans AndroidManifest.xml

Permission	Type	Usage
INTERNET	Normale	Appels API TMDB, Gemini
ACCESS_FINE_LOCATION	Dangereuse	GPS précis pour les cinémas
ACCESS_COARSE_LOCATION	Dangereuse	GPS approximatif
RECORD_AUDIO	Dangereuse	Recherche vocale
CAMERA	Dangereuse	Prise de photos

8.2 Gestion dynamique des permissions

La classe utilitaire `PermissionHelper` centralise la vérification et la demande de permissions à l'exécution, conformément aux exigences Android 6.0+.

Chapitre 9

Guide d'Installation

9.1 Prérequis

- Android Studio Hedgehog ou supérieur
- JDK 8 minimum
- Compte Firebase (gratuit)
- Clé API TMDB (gratuite sur themoviedb.org)
- Clé API Google Cloud (Maps + Places)
- Clé API Google AI Studio (Gemini)

9.2 Étapes de configuration

1. Créer un projet Android dans Android Studio avec le package `com.cinemago`
2. Coller le fichier `build.gradle` et synchroniser
3. Créer le projet Firebase et télécharger `google-services.json` dans `/app/`
4. Remplacer les clés dans `Constants.java` :
 - `TMDB_API_KEY`
 - `GEMINI_API_KEY`
 - `GOOGLE_MAPS_API_KEY` </itemize
 - Activer dans Firebase : Authentication (Email/Password), Firestore, Storage
 - Activer dans Google Cloud : Maps SDK Android, Places API (New)
 - Créer les icônes vectorielles via *File > New > Vector Asset*
 - Lancer l'application

9.3 Variables de configuration

```
1 public class Constants {  
2     // Remplacer par vos vraies clés  
3     public static final String TMDB_API_KEY =  
4         "VOTRE_CLE_TMDB_ICI";  
5     public static final String GEMINI_API_KEY =
```

```
6         "VOTRE_CLE_GEMINI_ICI";
7     public static final String GOOGLE_MAPS_API_KEY =
8         "VOTRE_CLE_MAPS_ICI";
9
10    // URLs de base (ne pas modifier)
11    public static final String TMDB_BASE_URL =
12        "https://api.themoviedb.org/3/";
13 }
```

Listing 9.1 – Fichier Constants.java — clés à remplacer

Chapitre 10

Conclusion et Perspectives

10.1 Bilan du projet

CinemaGo est une application Android complète qui démontre l'intégration réussie de nombreuses technologies modernes dans un cadre étudiant. Le projet couvre l'ensemble du cycle de développement mobile : authentification, consommation d'API, géolocalisation, intelligence artificielle, caméra et persistance des données.

Points forts du projet

- Architecture claire et séparée en couches
- 8 fonctionnalités majeures implémentées
- Code Java 100% sans Kotlin
- Thème sombre moderne et professionnel
- Singleton pattern pour Firebase
- Gestion des erreurs dans tous les callbacks

10.2 Perspectives d'amélioration

Les améliorations possibles pour une version future incluent :

- Mode hors-ligne** : Mise en cache locale avec Room Database
- Notifications Push** : Firebase Cloud Messaging pour les nouveautés
- Bandes-annonces** : Intégration YouTube API dans le détail film
- Système de notes** : Permettre à l'utilisateur de noter les films
- Partage social** : Partager un film sur les réseaux sociaux
- Mode sombre/clair** : Basculement de thème dynamique
- Multi-langue** : Localisation complète de l'interface

10.3 Répartition du code

TABLE 10.1 – Estimation de la répartition du code par package

Package	Fichiers Java	Fichiers XML
activities	6	6
fragments	5	5
adapters	4	5
models	5	0
api	2	0
firebase	1	0
chatbot	1	0
utils	4	0
Total	28	16

Références

- Documentation officielle Android : <https://developer.android.com/docs>
- API TMDb : <https://developers.themoviedb.org/3>
- Firebase Documentation : <https://firebase.google.com/docs>
- Google Maps Android SDK : <https://developers.google.com/maps/documentation/android-sdk>
- Google AI Gemini : <https://ai.google.dev/>
- Retrofit par Square : <https://square.github.io/retrofit/>
- CameraX : <https://developer.android.com/training/camerax>
- Glide Image Library : <https://github.com/bumptech/glide>